

تمرین برنامه نویسی شماره 4

سید علیرضا میرشفیعی

۸۱۰۱۰۱۵۳۲

فاز صفر

معماری کلی پروژه:

- **packet.hpp**: شامل تعریف ساختار فشرده هدر (با ویژگی `__attribute__((packed))`)، ساختار بسته (Packet) و توابع کمکی مانند محاسبه Checksum و سریال سازی داده ها.
- **sender.cpp**: شامل پیاده سازی فرستنده Mini-TCP، ماشین حالت کنترل ازدحام (Slow Start، Congestion Avoidance و Fast Recovery/Fast Retransmit)، مدیریت پنجره لغزان و پردازش ACK های دریافتی.
- **receiver.cpp**: شامل پیاده سازی گیرنده Mini-TCP برای دریافت بسته ها، شبیه سازی تأخیر و افت بسته ها (Loss & Delay)، مرتب سازی و ذخیره داده ها در فایل خروجی و ارسال ACK تجمعی (Cumulative ACK).
- **logger.hpp**: شامل کلاس ثبت وقایع با دقت میلی ثانیه برای ثبت رویدادهای مهم مانند ارسال و دریافت بسته ها، Retransmission، Timeout و تغییرات پنجره ازدحام.
- **Makefile**: شامل دستورات لازم برای کامپایل پروژه و اجرای خودکار سناریوهای مختلف آزمایش، همراه با تولید فایل های لاگ و خروجی های مورد نیاز.

توضیحات Header:

- **Seq_num**: برای مشخص کردن ترتیب قطعات داده ارسال شده استفاده می شود تا گیرنده بتواند فایل را به درستی و به ترتیب اولیه بازسازی کند.
- **ack_num**: برای اعلام دریافت موفقیت آمیز بسته ها به فرستنده کاربرد دارد. در فازهای بعدی که ACK تجمعی (Cumulative) پیاده سازی می شود، این فیلد نشان دهنده شماره بسته بعدی است که گیرنده انتظار دارد دریافت کند.
- **Length**: طول دقیق داده های قرار گرفته در بخش Payload بسته را مشخص می کند. این فیلد برای مواقعی که طول داده کمتر از حداکثر ظرفیت (Max Payload) است (مثلاً در انتهای فایل)، ضروری است.
- **Checksum**: برای تشخیص خطاهایی که ممکن است در حین انتقال بسته در شبکه رخ دهد استفاده می شود. فرستنده این مقدار را محاسبه کرده و گیرنده پس از دریافت، دوباره آن را محاسبه و بررسی می کند.
- **Flags**: برای تعیین نوع بسته کاربرد دارد:
 - **FLAG_DATA**: نشان می دهد بسته حاوی داده است.
 - **FLAG_ACK**: نشان دهنده بسته تایید (Acknowledgment) است.
 - **FLAG_FIN**: برای اعلام پایان ارتباط و اتمام ارسال فایل استفاده می شود.

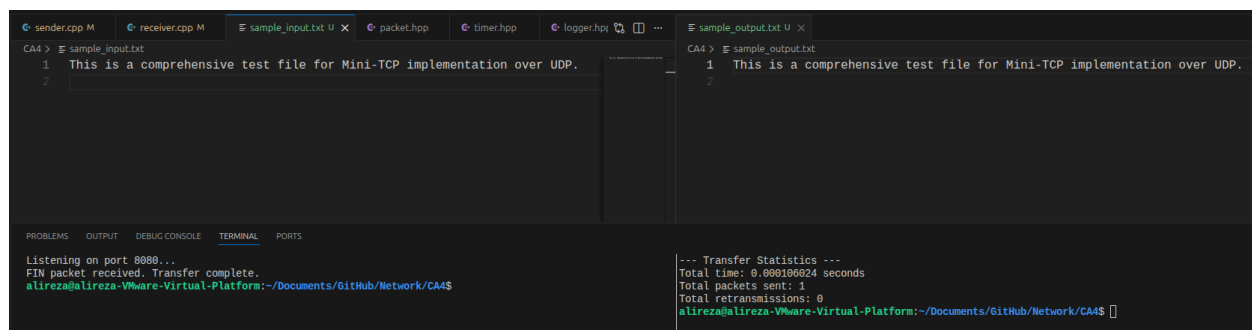
فاز یک

نحوه کار پروتکل Stop-and-Wait

در این روش، فرستنده در هر لحظه فقط یک بسته (Packet) ارسال می‌کند و منتظر می‌ماند (Stop). تا زمانی که گیرنده پیام تایید (ACK) آن بسته را ارسال نکند، فرستنده بسته بعدی را به هیچ‌وجه درون شبکه تزریق نمی‌کند. اگر در یک بازه زمانی مشخص (Timeout) تاییدیه به دست فرستنده نرسد، برنامه فرض می‌کند بسته یا ACK در شبکه گم شده و همان بسته را مجدداً ارسال (Retransmit) می‌کند.

تست اجرا

- **صحت انتقال:** همان‌طور که در تصویر مشخص است، متن درون sample_input.txt کلمه‌به‌کلمه و دقیق در sample_output.txt نوشته شده است. این یعنی دریافت، بررسی Checksum و نوشتن فایل در گیرنده کاملاً بی‌نقص بوده است.
- **آمار انتقال (Statistics):**
 - **Total packets sent: 1** : چون فایل آزمایشی بسیار کوچک (تنها یک خط متن) است، حجم آن کمتر از ظرفیت یک Payload (مثلاً ۱۰۲۴ بایت) بوده و کل فایل تنها در یک بسته (Packet) بسته‌بندی و ارسال شده است.
 - **Total retransmissions: 0** : چون برنامه روی بستر Localhost سیستم اجرا شده و در این فاز هنوز هیچ افت (Loss) یا تاخیری (Delay) شبیه‌سازی نشده، بسته در همان تلاش اول با موفقیت به مقصد رسیده و تایید شده است.
 - **زمان انتقال:** انتقال در زمان بسیار ناچیزی (حدود ۰.۰۰۰۱ ثانیه) و بلافاصله پس از بسته شدن ارتباط با پرچم FIN به پایان رسیده است که برای یک بسته در محیط لوکال کاملاً منطقی است.



```
CA4 > sample_input.txt
1 This is a comprehensive test file for Mini-TCP implementation over UDP.
2

CA4 > sample_output.txt
1 This is a comprehensive test file for Mini-TCP implementation over UDP.
2

--- Transfer Statistics ---
Total time: 0.000106024 seconds
Total packets sent: 1
Total retransmissions: 0
alireza@alireza-Virtual-Platform:~/Documents/GitHub/Network/CA4$
```

فاز دوم

نحوه کار پروتکل Sliding Window

برخلاف روش Stop-and-Wait، در این روش فرستنده برای ارسال بسته بعدی منتظر دریافت ACK نمی‌ماند. فرستنده اجازه دارد چندین بسته را به صورت متوالی (به اندازه ظرفیت یک متغیر به نام window_size درون شبکه تزریق کند.

- **لغزیدن پنجره:** با دریافت تأییدیه (ACK) برای بسته‌های قدیمی‌تر، این پنجره رو به جلو می‌لغزد و اجازه ارسال بسته‌های جدیدتر صادر می‌شود.
- **تأییدیه تجمعی (Cumulative ACK):** گیرنده در جواب، شماره بسته بعدی که انتظار دارد را می‌فرستد. مثلاً اگر ACK شماره ۵ را بفرستد، یعنی بسته‌های ۰ تا ۴ را با موفقیت دریافت کرده است. این کار باعث استفاده بهینه از پهنای باند و افزایش شدید سرعت (Throughput) می‌شود.

تست اجرا

- **پرتاب اولیه (Burst):** در سمت فرستنده (ترمینال راست)، می‌بینیم که در همان لحظه اول بسته‌های 0 تا 4 پشت سر هم ارسال شده‌اند بدون اینکه منتظر ACK بمانند. این نشان می‌دهد که window_size روی عدد ۵ تنظیم شده است.
- **عملکرد لغزش (Sliding):** به محض اینکه فرستنده اولین تأییدیه (ACK for expected seq: 1) را دریافت می‌کند، پنجره یک واحد به جلو می‌لغزد و بلافاصله بسته 5 را ارسال می‌کند. این چرخه تا پایان فایل ادامه دارد.
- **تأییدیه‌های تجمعی گیرنده:** در سمت گیرنده (ترمینال چپ)، با دریافت هر بسته، بلافاصله ACK تجمعی برای شماره بعدی ارسال شده است.
- **آمار نهایی:** کل ۸ بسته با ۰ بار ارسال مجدد (Retransmission) منتقل شده‌اند. به دلیل استفاده از پنجره لغزان و پر ماندن لوله ارتباطی شبکه، زمان انتقال به شدت کاهش یافته و کل فایل در کسر بسیار کوچکی از ثانیه (۰.۰۰۰۵ ثانیه) منتقل شده است.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[03:04:13.871] Listening on port 8080...
[03:04:14.389] Received DATA packet, seq: 0
[03:04:14.389] Sent cumulative ACK for next expected seq: 1
[03:04:14.389] Received DATA packet, seq: 1
[03:04:14.389] Sent cumulative ACK for next expected seq: 2
[03:04:14.389] Received DATA packet, seq: 2
[03:04:14.389] Sent cumulative ACK for next expected seq: 3
[03:04:14.389] Received DATA packet, seq: 3
[03:04:14.389] Sent cumulative ACK for next expected seq: 4
[03:04:14.389] Received DATA packet, seq: 4
[03:04:14.389] Sent cumulative ACK for next expected seq: 5
[03:04:14.389] Received DATA packet, seq: 5
[03:04:14.389] Sent cumulative ACK for next expected seq: 6
[03:04:14.389] Received DATA packet, seq: 6
[03:04:14.389] Sent cumulative ACK for next expected seq: 7
[03:04:14.389] Received DATA packet, seq: 7
[03:04:14.389] Sent cumulative ACK for next expected seq: 8
[03:04:14.389] FIN packet received. Transfer complete.
alireza@alireza-Virtual-Platform:~/Documents/GitHub/Network/CA4$

[03:04:14.389] Starting Sliding Window transmission. Total packets: 8
[03:04:14.389] Sent DATA packet, seq: 0
[03:04:14.389] Sent DATA packet, seq: 1
[03:04:14.389] Sent DATA packet, seq: 2
[03:04:14.389] Sent DATA packet, seq: 3
[03:04:14.389] Sent DATA packet, seq: 4
[03:04:14.389] Received ACK for expected seq: 1
[03:04:14.389] Sent DATA packet, seq: 5
[03:04:14.389] Received ACK for expected seq: 2
[03:04:14.389] Sent DATA packet, seq: 6
[03:04:14.389] Received ACK for expected seq: 3
[03:04:14.389] Sent DATA packet, seq: 7
[03:04:14.389] Received ACK for expected seq: 4
[03:04:14.389] Received ACK for expected seq: 5
[03:04:14.389] Received ACK for expected seq: 6
[03:04:14.389] Received ACK for expected seq: 7
[03:04:14.389] Received ACK for expected seq: 8
[03:04:14.389] Sent FIN packet. Transfer complete.
[03:04:14.389] Total Retransmission events: 0
[03:04:14.390] Total transfer time: 0.000503 seconds
alireza@alireza-Virtual-Platform:~/Documents/GitHub/Network/CA4$
```

دلیل برتری Sliding Window نسبت به Stop-and-Wait

در پروتکل Stop-and-Wait، فرستنده در هر لحظه تنها یک بسته (Packet) ارسال می‌کند و تا زمانی که تأییدیه (ACK) آن را دریافت نکند، بسته بعدی را ارسال نخواهد کرد. در نتیجه، طی مدت رفت و برگشت بسته و ACK

Trip Time یا (RTT ، فرستنده بدون انجام هیچ فعالیتی منتظر می‌ماند. این انتظار باعث می‌شود بخش قابل توجهی از ظرفیت و پهنای باند شبکه بلااستفاده بماند و کارایی انتقال داده کاهش یابد.

در مقابل، پروتکل **Sliding Window** این محدودیت را برطرف می‌کند. در این روش، فرستنده می‌تواند به اندازه ظرفیت پنجره، چندین بسته را به‌صورت متوالی و بدون انتظار برای دریافت ACK هر بسته ارسال کند. بنابراین، در مدت زمان انتقال بسته‌ها و بازگشت ACK ها نیز داده‌های جدید در حال ارسال هستند و ارتباط شبکه به‌صورت پیوسته حفظ می‌شود. (Pipelining)

در نتیجه، استفاده از پهنای باند بهینه‌تر شده، زمان‌های بیکاری فرستنده کاهش می‌یابد و توان عملیاتی (Throughput) افزایش پیدا می‌کند. به همین دلیل، پروتکل Sliding Window به‌ویژه در شبکه‌هایی با تأخیر بیشتر یا حجم بالای داده، عملکرد بهتری نسبت به Stop-and-Wait داشته و زمان کلی انتقال فایل را کاهش می‌دهد.

فاز سوم

نحوه پیاده‌سازی Retransmission و Timeout

- **مدیریت زمان (Timeout):** در ساختار فرستنده، برای مدیریت زمان انتظار، به جای درگیری با Thread ها و تایمرهای دستی، از تابع سیستمی select روی سوکت UDP استفاده شده است. این تابع منتظر دریافت داده (ACK) می‌ماند و یک تایمر داخلی (در اینجا ۱ ثانیه) دارد. اگر پیش از رسیدن هرگونه پاسخی از گیرنده، این زمان به پایان برسد، تابع select مقدار صفر را برمی‌گرداند که نشان‌دهنده وقوع انقضای زمان یا Timeout است.
- **ارسال مجدد (Go-Back-N Retransmission):** به محض تشخیص Timeout ، فرستنده نتیجه می‌گیرد که بسته ارسال‌شده در شبکه گم شده است. در این حالت، متغیر next_seq (که نشان‌دهنده بسته بعدی برای ارسال است) مستقیماً به مقدار base (شماره قدیمی‌ترین بسته تأییدنشده) ریست می‌شود. با این کار، فرستنده تمام بسته‌های درون پنجره ارسال را که هنوز تأییدیه نگرفته‌اند، از ابتدا و به ترتیب مجدداً به درون شبکه تزریق می‌کند (Retransmit).

تست اجرا

- **شبیه‌سازی افت بسته (Packet Loss):** در لاگ گیرنده (سمت چپ) به وضوح می‌بینیم که برنامه به صورت مصنوعی بسته شماره ۷ را نادیده می‌گیرد. (SIMULATED LOSS: Dropped packet seq 7)
- **عملکرد تأییدیه تجمعی (Cumulative ACK):** از آنجا که بسته ۷ گم شده، گیرنده با دریافت بسته‌های بعدی (۸، ۹، ۱۰ و ۱۱) که خارج از ترتیب رسیده‌اند، از تأیید آن‌ها خودداری می‌کند. در عوض، برای هر کدام از آن‌ها، مجدداً پیام ACK for next expected seq: 7 را به فرستنده می‌فرستد تا به او بفهماند که همچنان منتظر بسته شماره ۷ است.
- **وقوع Timeout و ارسال مجدد:** فرستنده با دریافت این ACK های تکراری، تغییری در پنجره خود نمی‌دهد. در نهایت، با گذشت زمان تعیین‌شده، تایمر منقضی شده (Timeout occurred!) و فرستنده کل پنجره را از همان بسته شماره ۷ دوباره ارسال می‌کند. (Retransmitting window from seq: 7) همین روند کمی بعد برای بسته شماره ۸ نیز تکرار شده است.

- **آمار و توان عملیاتی (Throughput):** در آمار نهایی، ثبت ۳ رویداد Retransmission نشان می‌دهد که شبکه ۳ بار دچار وقفه کامل ۱ ثانیه‌ای شده است. این توقف‌ها باعث شده زمان انتقال به ۴.۳۳ ثانیه برسد و توان عملیاتی (Throughput) به شدت افت کرده و عدد ۳۷۱۵ بایت بر ثانیه را ثبت کند. این خروجی به شکل دقیقی نشان‌دهنده تاثیر مخرب افت بسته بر پروتکل‌هایی است که فقط به مکانیزم پایه Timeout وابسته‌اند.

```
[03:40:05.954] Received DATA packet, seq: 6
[03:40:06.005] Sent cumulative ACK for next expected seq: 7
[03:40:06.005] SIMULATED LOSS: Dropped packet seq 7
[03:40:06.005] Received DATA packet, seq: 8
[03:40:06.056] Sent cumulative ACK for next expected seq: 7
[03:40:06.056] Received DATA packet, seq: 9
[03:40:06.106] Sent cumulative ACK for next expected seq: 7
[03:40:06.106] Received DATA packet, seq: 10
[03:40:06.157] Sent cumulative ACK for next expected seq: 7
[03:40:06.157] SIMULATED LOSS: Dropped packet seq 11
[03:40:07.159] Received DATA packet, seq: 7
[03:40:07.210] Sent cumulative ACK for next expected seq: 8
[03:40:07.210] SIMULATED LOSS: Dropped packet seq 8
[03:40:07.210] Received DATA packet, seq: 9
[03:40:07.260] Sent cumulative ACK for next expected seq: 8
[03:40:07.261] Received DATA packet, seq: 10
[03:40:07.311] Sent cumulative ACK for next expected seq: 8
[03:40:07.311] Received DATA packet, seq: 11
[03:40:07.362] Sent cumulative ACK for next expected seq: 8
[03:40:07.363] Received DATA packet, seq: 12
[03:40:07.414] Sent cumulative ACK for next expected seq: 8
[03:40:08.415] Received DATA packet, seq: 8
[03:40:08.465] Sent cumulative ACK for next expected seq: 9
[03:40:08.465] Received DATA packet, seq: 9
[03:40:08.516] Sent cumulative ACK for next expected seq: 10
[03:40:08.516] Received DATA packet, seq: 10
[03:40:08.567] Sent cumulative ACK for next expected seq: 11
[03:40:08.567] Received DATA packet, seq: 11
[03:40:08.617] Sent cumulative ACK for next expected seq: 12
[03:40:08.618] Received DATA packet, seq: 12
[03:40:08.668] Sent cumulative ACK for next expected seq: 13
[03:40:08.670] Received DATA packet, seq: 13
[03:40:08.721] Sent cumulative ACK for next expected seq: 14
[03:40:08.721] Received DATA packet, seq: 14
[03:40:08.771] Sent cumulative ACK for next expected seq: 15
[03:40:08.771] Received DATA packet, seq: 15
[03:40:08.823] Sent cumulative ACK for next expected seq: 16
[03:40:08.824] FIN packet received. Transfer complete.
--- Receiver finished ---

[03:40:06.005] Received ACK for expected seq: 7
[03:40:06.005] Sent DATA packet, seq: 11
[03:40:06.056] Received ACK for expected seq: 7
[03:40:06.106] Received ACK for expected seq: 7
[03:40:06.157] Received ACK for expected seq: 7
[03:40:07.158] Timeout occurred! Retransmitting window from seq: 7
[03:40:07.159] Sent DATA packet, seq: 7
[03:40:07.159] Sent DATA packet, seq: 8
[03:40:07.159] Sent DATA packet, seq: 9
[03:40:07.159] Sent DATA packet, seq: 10
[03:40:07.159] Sent DATA packet, seq: 11
[03:40:07.209] Received ACK for expected seq: 8
[03:40:07.210] Sent DATA packet, seq: 12
[03:40:07.261] Received ACK for expected seq: 8
[03:40:07.311] Received ACK for expected seq: 8
[03:40:07.362] Received ACK for expected seq: 8
[03:40:07.413] Received ACK for expected seq: 8
[03:40:08.415] Timeout occurred! Retransmitting window from seq: 8
[03:40:08.415] Sent DATA packet, seq: 8
[03:40:08.415] Sent DATA packet, seq: 9
[03:40:08.415] Sent DATA packet, seq: 10
[03:40:08.415] Sent DATA packet, seq: 11
[03:40:08.415] Sent DATA packet, seq: 12
[03:40:08.465] Received ACK for expected seq: 9
[03:40:08.465] Sent DATA packet, seq: 13
[03:40:08.516] Received ACK for expected seq: 10
[03:40:08.516] Sent DATA packet, seq: 14
[03:40:08.567] Received ACK for expected seq: 11
[03:40:08.567] Sent DATA packet, seq: 15
[03:40:08.617] Received ACK for expected seq: 12
[03:40:08.668] Received ACK for expected seq: 13
[03:40:08.721] Received ACK for expected seq: 14
[03:40:08.771] Received ACK for expected seq: 15
[03:40:08.824] Received ACK for expected seq: 16
[03:40:08.824] Sent FIN packet. Transfer complete.
[03:40:08.824] --- STATISTICS ---
[03:40:08.824] Total Retransmission events: 3
[03:40:08.824] Total transfer time: 4.330493 seconds
[03:40:08.824] Throughput: 3715.974254 Bytes/sec
--- Sender finished ---
```

تحلیل اثر Loss و Delay بر عملکرد پروتکل

با افزایش **Loss**، تعداد بیشتری از بسته‌ها از بین می‌روند و فرستنده پس از پایان زمان **Timeout**، مطابق مکانیزم **Go-Back-N** بسته‌های تأیید نشده را دوباره ارسال می‌کند. در نتیجه، تعداد **Retransmission**‌ها افزایش یافته و بخشی از پهنای باند صرف ارسال مجدد داده‌ها می‌شود.

از طرف دیگر، افزایش **Delay** باعث طولانی‌تر شدن زمان دریافت **ACK** و افزایش زمان کلی انتقال فایل می‌شود.

با توجه به رابطه $\text{Throughput} = \text{Data} / \text{Time}$ ، افزایش زمان انتقال باعث کاهش **Throughput** می‌شود. بنابراین، افزایش **Loss** و **Delay** هر دو موجب کاهش کارایی شبکه و افت توان عملیاتی خواهند شد.

فاز چهارم

نحوه پیاده‌سازی Slow Start و Congestion Avoidance

مکانیزم کنترل ازدحام (TCP Congestion Control) با استفاده از سه متغیر اصلی مدیریت می‌شود: پنجره ازدحام (**cwnd**) با مقدار اولیه ۱، آستانه شروع آهسته (**ssthresh**) با یک مقدار اولیه بزرگ (مثلاً ۵۰ یا ۶۴)، و شمارنده تأییدیه‌ها. پیاده‌سازی آن به شرح زیر است:

- **شروع آهسته (Slow Start):** تا زمانی که $cwnd < ssthresh$ باشد، فرستنده در فاز Slow Start قرار دارد. در این فاز، با دریافت هر ACK معتبر و غیرتکراری، مقدار پنجره ازدحام یک واحد افزایش می‌یابد ($cwnd += 1$). این رفتار باعث می‌شود که اندازه پنجره در هر زمان رفت و برگشت (RTT) به صورت نمایی دوبرابر شود تا سریعاً از ظرفیت خالی شبکه استفاده کند.
- **جلوگیری از ازدحام (Congestion Avoidance):** به محض اینکه $cwnd \geq ssthresh$ شود، فرستنده وارد فاز رشد خطی می‌شود. در این فاز، برای جلوگیری از سرریز ناگهانی بافرهای شبکه، با دریافت هر ACK معتبر، مقدار پنجره به صورت محتاطانه‌تر و بر اساس رابطه $cwnd += 1.0 / cwnd$ افزایش می‌یابد. به این ترتیب، در هر RTT پنجره حداکثر یک واحد رشد می‌کند.
- **برخورد با رخداد Timeout:** در صورت منقضی شدن تایمر (و وقوع Timeout)، فرستنده فرض می‌کند شبکه دچار احتقان شدید شده است. در نتیجه:
 - a. مقدار آستانه به نصف پنجره فعلی کاهش می‌یابد: $ssthresh = \max(2.0, cwnd / 2.0)$
 - b. پنجره ازدحام به مقدار اولیه خود سقوط می‌کند: $cwnd = 1.0$
 - c. فرستنده مجدداً کار خود را از فاز Slow Start آغاز می‌کند.

تست اجرا

با بررسی تصویر ترمینال فرستنده (سمت چپ)، فرآیند کنترل ازدحام به صورت پویا و کاملاً منطبق بر تئوری شبکه‌ها دیده می‌شود:

- **وقوع Timeout و به‌روزرسانی پارامترها:** در لاگ عبارت `Timeout occurred!` ثبت شده است. بلافاصله پس از این رخداد، سیستم مقدار آستانه را به عدد ۲ کاهش داده است (`ssthresh updated to: 2`) و اندازه پنجره ازدحام را مجدداً به ۱ بازگردانده است (`cwnd reset to 1`). این رفتار گامی پیشگیرانه برای تخلیه بافرهای شبکه است.
- **انتقال حالت پویای پروتکل:**
 - پس از ریست شدن، فرستنده کار خود را در فاز Slow Start آغاز می‌کند و بلافاصله پس از دریافت ACK، پنجره را به سرعت افزایش می‌دهد. (`Updating CWND (Slow Start) -> Old: 1, New: 2`)
 - به محض اینکه مقدار پنجره به مرز آستانه (`ssthresh = 2`) می‌رسد، سیستم تغییر وضعیت داده و از این لحظه به بعد با فرمول رشد خطی می‌کند (`Updating CWND (Congestion Avoidance) -> Old: 2, New: 2.something`). شروع آهسته و جلوگیری از ازدحام است.
- **شاخص‌های عملکردی نهایی (Metrics):** ثبت زمان انتقال ۴.۲۶ ثانیه و توان عملیاتی ۳۷۷۰ بایت بر ثانیه (با وجود اعمال ۳ بار رخداد Timeout در سناریو (تایید می‌کند که سیستم پس از هر ریزش و قطعی موقت در کانال انتقال، به سرعت خود را بازیابی کرده و بدون قفل شدن ارتباط، ارسال داده‌ها را با نرخ متناسب با کشش شبکه ادامه داده است).

```

CA4 > Junk > Logs > cwnd.csv > data
1 Time(s),Cwnd
2 6.3e-08,1
3 0.0559854,2
4 0.106902,3
5 0.157627,4
6 0.208537,5
7 1.3636,1
8 1.41519,2
9 1.46835,2.5
10 1.51886,3
11 1.57029,3.33333
12 1.62332,3.66667
13 2.72686,1
14 2.77792,2
15 2.82887,2.5
16 2.88036,3
17 2.9311,3.33333
18 2.98171,3.66667
19 3.03276,4
20 3.08286,4.25
21

CA4 > Junk > Logs > events.log
51 [05:16:28.374] Received ACK for expected seq: 11
52 [05:16:28.374] Congestion Avoidance -> cwnd increased to 2.500000
53 [05:16:28.375] Sent DATA packet, seq: 12
54 [05:16:28.425] Received ACK for expected seq: 12
55 [05:16:28.426] Congestion Avoidance -> cwnd increased to 3.000000
56 [05:16:28.426] Sent DATA packet, seq: 13
57 [05:16:28.426] Sent DATA packet, seq: 14
58 [05:16:28.476] Received ACK for expected seq: 13
59 [05:16:28.477] Congestion Avoidance -> cwnd increased to 3.333333
60 [05:16:28.477] Sent DATA packet, seq: 15
61 [05:16:28.527] Received ACK for expected seq: 14
62 [05:16:28.527] Congestion Avoidance -> cwnd increased to 3.666667
63 [05:16:28.578] Received ACK for expected seq: 15
64 [05:16:28.578] Congestion Avoidance -> cwnd increased to 4.000000
65 [05:16:28.628] Received ACK for expected seq: 16
66 [05:16:28.628] Congestion Avoidance -> cwnd increased to 4.250000
67 [05:16:28.629] Sent FIN packet. Transfer complete.
68 [05:16:28.629] --- STATISTICS ---
69 [05:16:28.629] Total Retransmission events: 2
70 [05:16:28.629] Total transfer time: 3.082867 seconds
71 [05:16:28.629] Throughput: 5219.815651 Bytes/sec

```

واکنش TCP به Timeout

وقتی در TCP یک **Timeout** رخ می‌دهد، پروتکل فرض می‌کند که شبکه دچار **ازدحام شدید (Severe Congestion)** شده است. در این حالت، چون هیچ **ACK** برای بسته ارسال شده دریافت نشده، TCP مقدار **cwnd** را به ۱ کاهش می‌دهد تا فشار روی شبکه کم شده و بافرهای مسیر فرصت تخلیه پیدا کنند.

سپس ارتباط دوباره از مرحله **Slow Start** آغاز می‌شود. در این مرحله، **cwnd** به صورت نمایی افزایش می‌یابد تا ظرفیت آزاد شبکه دوباره شناسایی شود. این افزایش تا رسیدن به مقدار **ssthresh** (که معمولاً نصف مقدار قبلی است) ادامه پیدا می‌کند و پس از آن رشد پنجره به صورت تدریجی انجام می‌شود تا از بروز مجدد ازدحام جلوگیری شود.

فاز پنجم

تحلیل خروجی ها

آزمایش اول: شبکه بدون افت (Loss = 0%, Delay = 0ms)

- **نحوه رشد cwnd:** همان‌طور که در نمودار تست ۱ (خط آبی) مشخص است، فرستنده کار خود را با فاز **Slow Start** آغاز کرده و پنجره به صورت نمایی رشد می‌کند. پس از رسیدن به آستانه (**ssthresh**)، پروتکل به صورت هوشمندانه وارد فاز **Congestion Avoidance** شده و شیب رشد آن خطی و ملایم‌تر می‌شود. از آنجا که هیچ بسته‌ای گم نمی‌شود، پنجره به طور پیوسته تا پایان انتقال فایل (حدود ۱۸ پکت) بزرگ می‌شود.
- **توان عملیاتی (Throughput) نهایی:** به دلیل عدم وجود تاخیر و افت بسته، سیستم از تمام پهنای باند شبکه محلی (Localhost) استفاده می‌کند. نمودار **Throughput** (تست ۱) نشان می‌دهد که توان عملیاتی بسیار بالا و در حدود ۱۰ مگابایت بر ثانیه (10^7 B/s) است.

آزمایش دوم: شبکه با افت متوسط (Loss = 10%, Delay = 50ms)

- **رخداد: Timeout** با اعمال ۱۰٪ افت بسته، سیستم به طور مکرر دچار مفقود شدن بسته‌ها می‌شود. با هر بار گم شدن بسته، فرستنده تاییدیه (ACK) آن را دریافت نمی‌کند و پس از گذشت زمان تعیین شده، پدیده Timeout رخ می‌دهد.
- **نحوه کاهش: cwnd** نمودار تست ۲ (خط زرد/نارنجی) الگوی معروف "دندانه‌اره‌ای (Saw-tooth)" پروتکل TCP را به وضوح نشان می‌دهد. با هر بار وقوع Timeout، پروتکل فرض می‌کند شبکه دچار ازدحام شده است؛ در نتیجه بلافاصله مقدار cwnd را به ۱ سقوط می‌دهد (Drop to 1) و آستانه (ssthresh) را به نصف پنجره فعلی کاهش می‌دهد. سپس دوباره تلاش می‌کند با Slow Start ظرفیت شبکه را بسنجد. این ریزش‌ها به طور مداوم در طول انتقال تکرار شده‌اند.

آزمایش سوم: شبکه با افت زیاد (Loss = 20%, Delay = 100ms)

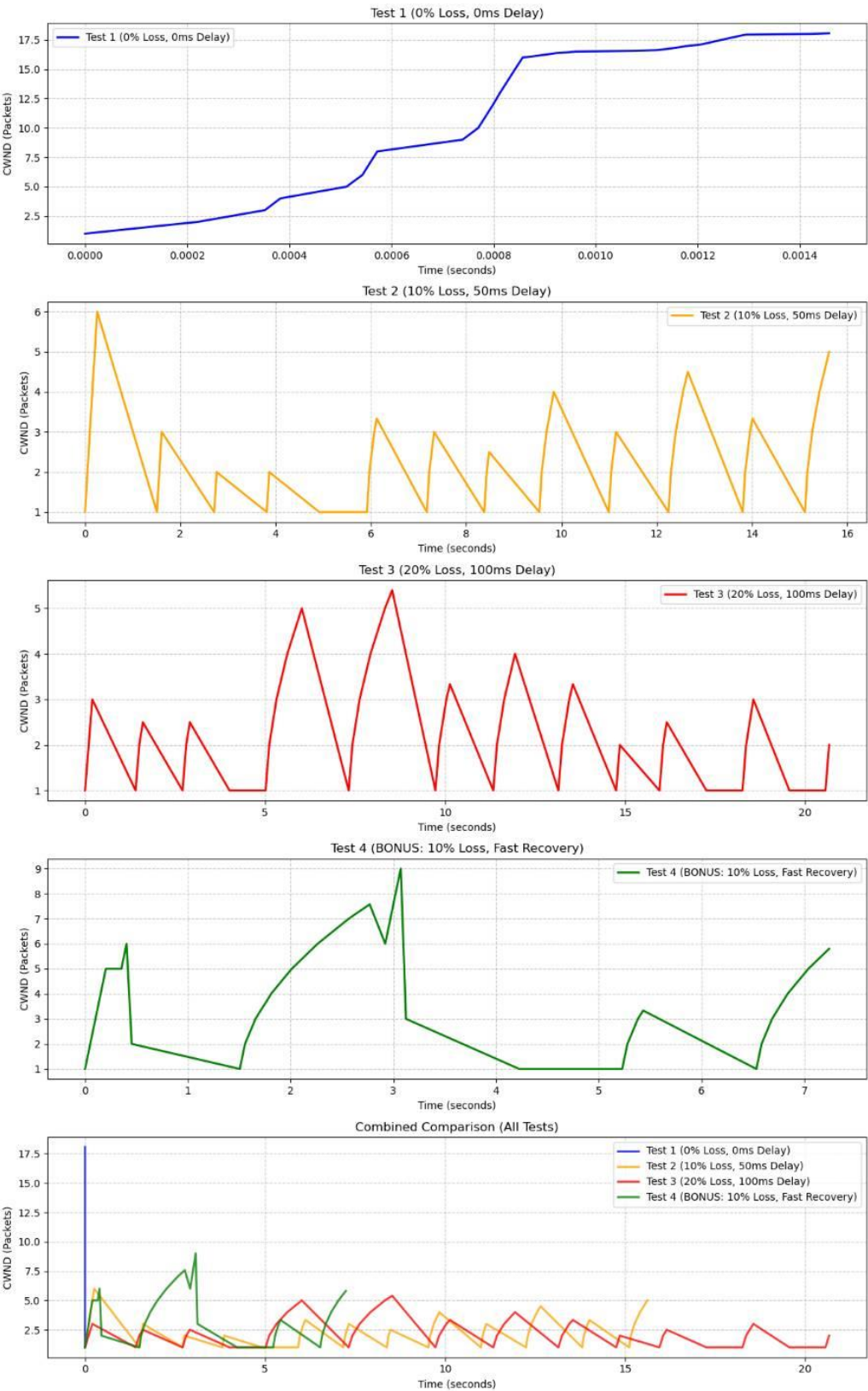
- **صحت انتقال فایل:** با وجود افت بسیار شدید ۲۰٪ و تاخیر بالا، پروتکل توانسته است به لطف پیاده‌سازی دقیق مکانیزم‌های Retransmission و Cumulative ACK، فایل را به صورت کاملاً سالم و بدون نقص به گیرنده منتقل کند. این نشان‌دهنده پایداری (Reliability) بالای این پیاده‌سازی است.
- **کاهش: Throughput** نمودار تست ۳ (خط قرمز) نشان می‌دهد که توان عملیاتی به شدت آسیب دیده و دچار نوسانات شدید بین ۲۰۰۰ تا ۱۴۰۰۰ بایت بر ثانیه شده است. افت مکرر بسته‌ها باعث شده تا فرستنده بیشتر زمان خود را در وضعیت Timeout و ارسال مجدد بسته‌های قدیمی (Go-Back-N) بگذراند تا ارسال داده‌های جدید.

تحلیل نهایی: اثر افزایش Loss و Delay بر عملکرد پروتکل

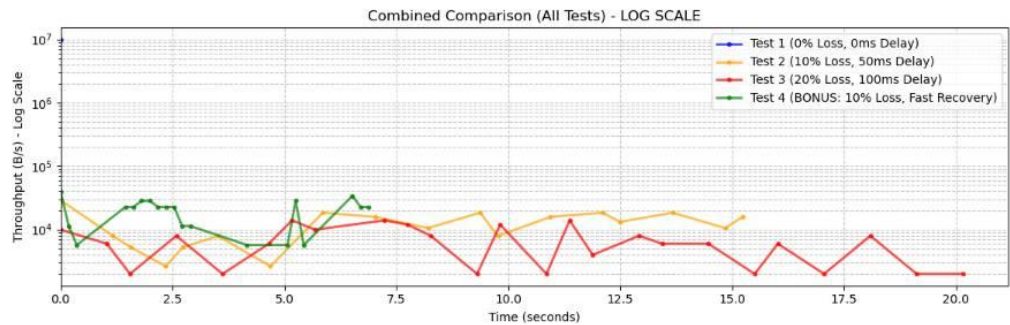
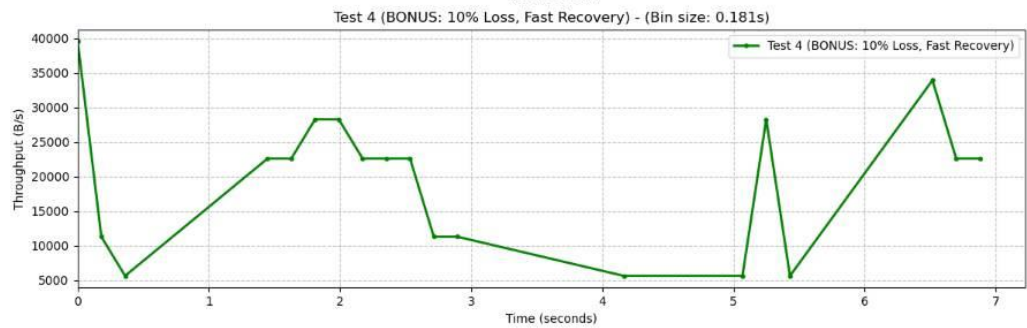
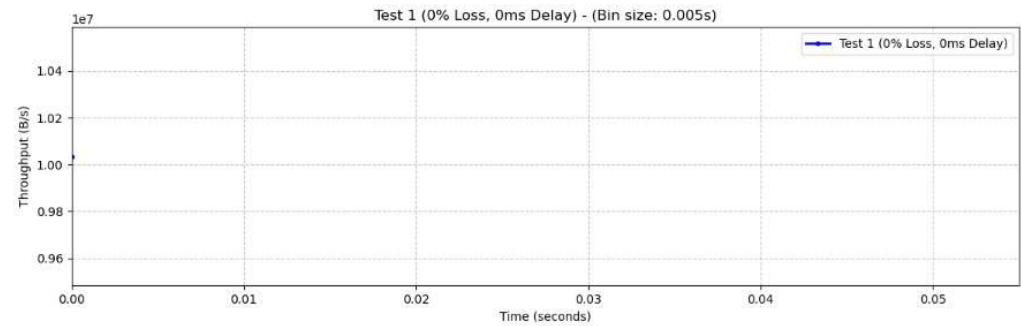
با مقایسه همزمان نمودارها (Combined Comparison)، می‌توان نتیجه‌گیری‌های زیر را استخراج کرد:

۱. **اثر Loss بر کارایی شبکه:** با افزایش درصد افت بسته (از ۰٪ به ۱۰٪ و ۲۰٪)، تعداد دفعات Timeout به شدت بالا می‌رود. این امر باعث می‌شود اندازه پنجره (cwnd) نتواند به اندازه کافی بزرگ شود و مدام به ۱ ریست گردد. کوچک ماندن پنجره به معنای ارسال بسته‌های کمتر در هر RTT و در نتیجه سقوط شدید توان عملیاتی (Throughput) است.
۲. **اثر Delay بر سرعت رشد: تاخیر (Delay)** مستقیماً روی زمان رفت و برگشت (RTT) تأثیر می‌گذارد. در تست ۳ که تاخیر ۱۰۰ms است، حتی در فاز Slow Start نیز مدت زمان زیادی طول می‌کشد تا تاییدیه‌ها به فرستنده برسند و پنجره یک واحد بزرگتر شود. این کندی در دریافت ACK‌ها، باعث طولانی‌تر شدن زمان کل انتقال فایل و افت ملموس Throughput می‌شود.
۳. **تغییر رویکرد مصرف پهنای باند:** در شبکه بدون افت، پهنای باند صرف انتقال داده‌های مفید می‌شود؛ اما در شبکه‌های دارای Loss بالا، درصد عظیمی از پهنای باند و زمان شبکه صرف ارسال مجدد (Retransmission) داده‌های تکراری می‌گردد که راندمان سیستم را به شدت کاهش می‌دهد.

Congestion Window (cwnd) over Time



Throughput over Time



فاز امتیازی

مفهوم cwnd و ssthresh

در پروتکل TCP ، مدیریت پهنای باند و جلوگیری از خفگی شبکه توسط دو متغیر کلیدی کنترل می‌شود:

- **متغیر: cwnd (Congestion Window)** این متغیر مشخص می‌کند که فرستنده در هر لحظه، حداکثر چند بسته را می‌تواند بدون انتظار برای دریافت تأییدیه به درون شبکه بفرستد. این مقدار پویا است و بر اساس شرایط شبکه بزرگ یا کوچک می‌شود.
- **متغیر: ssthresh (Slow Start Threshold)** یک حد آستانه است که فاز کاری کنترل ازدحام را تعیین می‌کند. تا زمانی که مقدار cwnd کمتر از ssthresh باشد، پروتکل در فاز شروع آهسته (Slow Start) قرار دارد و به صورت نمایی رشد می‌کند. وقتی cwnd به این آستانه می‌رسد یا از آن عبور می‌کند، پروتکل وارد فاز اجتناب از ازدحام (Congestion Avoidance) شده و محتاطانه‌تر (رشد خطی) عمل می‌کند. در زمان وقوع قطعی یا Timeout ، این آستانه به نصف مقدار فعلی cwnd کاهش می‌یابد.

تحلیل خروجی

بر اساس خروجی‌های ثبت شده در نمودار ترکیبی (Bonus Phase Analysis) ، می‌توان عملکرد مکانیزم‌های بازیابی سریع را به شرح زیر تحلیل کرد:

- **رسم نمودار cwnd و مشخص کردن نقاط: Fast Retransmit** در نمودار سمت چپ، روند تغییرات پنجره ازدحام رسم شده است. در پروتکل TCP اگر فرستنده چند ACK تکراری دریافت کند، پیش از وقوع Timeout حدس می‌زند که یک بسته از بین رفته است. به این مکانیزم Fast Retransmit گفته می‌شود. این رویدادها با علامت **ضربدر قرمز (X)** به دقت روی نمودار مارک شده‌اند. در این نقاط، سیستم به جای سقوط آزاد پنجره به مقدار ۱، به لطف مکانیزم Fast Recovery با یک افت کنترل‌شده مواجه شده و فرآیند ارسال را سریعاً بازیابی می‌کند.
- **مقایسه تعداد Timeout ها:** بر اساس نمودار میله‌ای وسط، در سناریوی تست با شرایط یکسان، حالت استاندارد 28 بار دچار Timeout شده است. اما با فعال‌سازی حالت امتیازی (Fast Mode) ، تعداد Timeout ها به 21 کاهش یافته است. دلیل این امر آن است که سیستم در بسیاری از موارد توانسته است گم شدن بسته را از طریق ACK های تکراری زودتر تشخیص دهد و پیش از منقضی شدن کامل تایمر، بسته را بازپخش کند.
- **مقایسه زمان انتقال:** در نمودار سمت راست، تأثیر مستقیم این مکانیزم‌ها بر سرعت شبکه مشهود است. زمان کل انتقال فایل در حالت استاندارد 37.07s بوده، در حالی که در حالت Fast Mode این زمان به 30.57s کاهش یافته است. کاهش تعداد توقف‌های کامل ناشی از Timeout و جلوگیری از شروع مجدد از مرحله Slow Start (به لطف Fast Recovery) ، باعث شده تا پهنای باند با بهرهوری بسیار بالاتری مصرف شود و زمان انتقال حدود ۱۷% بهبود یابد.

Bonus Phase Analysis: Fast Retransmit & Fast Recovery

